Conditions and Branching

Estimated time needed: 10 minutes

Objective:

In this reading, you'll learn about:

1. Comparison operators
2. Branching
3. Logical operators

1. Comparison operations

Comparison operations are essential in programming. They help compare values and make decisions based on the results.

Equality operator

The equality operator == checks if two values are equal. For example, in Python:

```
age = 25
if age == 25:
    print("You are 25 years old.")
```

Here, the code checks if the variable age is equal to 25 and prints a message accordingly.

Inequality operator

The inequality operator `!=` checks if two values are not equal:

```
if age != 30:
    print("You are not 30 years old.")
```

Here, the code checks if the variable age is not equal to 30 and prints a message accordingly.

Greater than and less than

You can also compare if one value is greater than another.

```
if age>= 20:
    Print("Yes, the Age is greater than 20")
```

Here, the code checks if the variable age is greater than or equal to 20 and prints a message accordingly.

2. Branching

Branching is like making decisions in your program based on conditions. Think of it as real-life choices.

The IF statement

Consider a real-life scenario of entering a bar. If you're above a certain age, you can enter; otherwise, you cannot.

```
age = 20
if age >= 21:
    print("You can enter the bar.")
else:
    print("Sorry, you cannot enter.")
```

Here, you are using the `if` statement to make a decision based on the age variable.

The ELIF Statement

Sometimes, there are multiple conditions to check. For example, if you're not old enough for the bar, you can go to a movie instead.

```
if age >= 21:
    print("You can enter the bar.")
elif age >= 18:
    print("You can watch a movie.")
else:
    print("Sorry, you cannot do either.")
```

Real-life example: Automated Teller Machine (ATM)

When a user interacts with an ATM, the software in the ATM can use branching to make decisions based on the user's input. For example, if the user selects "Withdraw Cash" the ATM can branch into different denominations of bills to dispense based on the amount requested.

```
user_choice = "Withdraw Cash"
if user_choice == "Withdraw Cash":
    amount = int(input("Enter the amount to withdraw: "))
    if amount % 10 == 0:
        print("Amount dispensed: ", amount)
    else:
        print("Please enter a multiple of 10.")
else:
    print("Thank you for using the ATM.")
```

3. Logical operators

Logical operators help combine and manipulate conditions.

The NOT operator

Real-life example: Notification settings

In a smartphone's notification settings, you can use the NOT operator to control when to send notifications. For example, you might only want to receive notifications when your phone is not in "Do Not Disturb" mode.

The not operator negates a condition.

```
is_do_not_disturb = True
if not is_do_not_disturb:
    send_notification("New message received")
```

The AND operator

Real-life example: Access control

In a secure facility, you can use the AND operator to check multiple conditions for access. To open a high-security door, a person might need both a valid ID card and a matching fingerprint.

The AND operator checks if all required conditions are true, like needing both keys to open a safe.

```
has_valid_id_card = True
has_matching_fingerprint = True
if has_valid_id_card and has_matching_fingerprint:
    open_high_security_door()
```

The OR operator

Real-life example: Movie night decision

When planning a movie night with friends, you can use the OR operator to decide on a movie genre. You'll choose a movie if at least one person is interested.

The OR operator checks if at least one condition is true. It's like choosing between different movies to watch.

```
friend1_likes_comedy = True
friend2_likes_action = False
friend3_likes_drama = False
if friend1_likes_comedy or friend2_likes_action or friend3_likes_drama:
    choose a movie()
```

Summary

In this reading, you delved into the most frequently used operator and the concept of conditional branching, which encompasses the utilization of `if` statements and `if-else` statements.

Author

Akansha Yadav